1. Write a Python program that uses a count-controlled loop (`for` loop) to ask the user to enter 5 test marks (integers). The program must calculate and output the total and the average mark.

```
total = 0

for i in range(5):
    mark = int(input("Enter mark: "))
    total = total + mark

average = total / 5

print("Total:", total)
print("Average:", average)
```

2. Write a Python program using a conditional loop (`while` loop) that repeatedly asks the user to enter a positive integer.
    - Entering `-1` serves as the rogue value (sentinel) to stop the loop.
    - Any other negative number or zero should display an error message "`Invalid input, must be positive or -1 to stop`" and not be processed.
    - After the user enters -1, output the count of valid numbers entered.

```
count = 0
number = int(input("Enter a positive number (-1 to stop): "))

while number != -1:
    if number > 0:
        count = count + 1
    else:
        print("Invalid input, must be positive or -1 to stop")

     number = int(input("Enter a positive number (-1 to stop):
"))

print("You entered", count, "valid numbers.")
```

3. A teacher wants to find the highest score achieved in an exam. Write a Python program that:
    - Prompts the teacher to enter how many student scores will be recorded.
    - Uses a loop to input each score.
    - Finds and prints both the highest score and lowest score entered.

```
num_scores = int(input("How many scores? "))

# Read first value to seed extremes
first_score = int(input("Enter score: "))
highest = first_score
lowest = first_score

# Loop for the remaining scores
for i in range(num_scores - 1):
    score = int(input("Enter score: "))
    if score > highest:
        highest = score
    if score < lowest:
        lowest = score

print("Highest score:", highest)
print("Lowest score:", lowest)
```

4. Write a Python program that initializes a 1-D array (list) containing 6 test scores: `[45, 62, 78, 55, 90, 68]`.

   Your program must:

   - Use a count-controlled loop (`for` loop) to iterate through the array.
   - Calculate the total sum of all the marks in the array.
   - Calculate and display the average mark.

```
scores = [45, 62, 78, 55, 90, 68]

total = 0

for i in range(len(scores)):
    total = total + scores[i]

average = total / len(scores)

print("Total:", total)
print("Average:", round(average, 2))
```

5. A weather station records the daily temperature for one week using a 1-D array:

```
temperatures = [28, 31, 29, 32, 30, 33, 27]
```

   Write a Python program that:

- Loops through the array.
- Count how many days recorded a temperature strictly higher than 30°C.
- Outputs the total count with an appropriate message.

```
temperatures = [28, 31, 29, 32, 30, 33, 27]

count = 0

for i in range(len(temperatures)):
    if temperatures[i] > 30:
        count = count + 1

print("Days with temperature above 30C:", count)
```

Linear search

6. Write a Python program that pre-populates a 1-D array with 5 student names:

```
names = ["Ali", "Sarah", "Dev", "Mei", "Carlos"]
```

The program must:

- Ask the user to enter a name to search for.
- Use a loop and a boolean flag (`found = False`) to perform a linear search through the array.
- If the name is present, display `"Found at position "` followed by its index position (0 to 4).
- If the name is not in the array after checking every element, display `"Name not found"`

```
names = ["Ali", "Sarah", "Dev", "Mei", "Carlos"]

search_name = input("Enter name to search: ")

found = False
position = -1

for i in range(len(names)):
    if names[i] == search_name:
        found = True
        position = i
        break

if found:
    print("Found at position", position)
else:
    print("Name not found")
```

Question 7

Write a Pseudocode and Python program that uses the **Bubble Sort** algorithm to sort a 1-D array of 5 integers in ascending order (smallest to largest).

Pre-populated array: `numbers = [42, 15, 8, 23, 4]`

Your program must:

- Use nested loops to compare adjacent elements.
- Use a temporary variable (`temp`) to swap elements if they are in the wrong order.
- Print the sorted array after sorting is complete.

Pseudocode	Python

DECLARE numbers : ARRAY[0:4] OF INTEGER
DECLARE temp : INTEGER
DECLARE i : INTEGER
DECLARE j : INTEGER
 	
numbers[0] ← 42
numbers[1] ← 15
numbers[2] ← 8
numbers[3] ← 23
numbers[4] ← 4
 	
FOR i ← 0 TO 3
    FOR j ← 0 TO 3 - i
        IF numbers[j] > numbers[j + 1]
THEN
            temp ← numbers[j]
            numbers[j] ← numbers[j + 1]
            numbers[j + 1] ← temp
        ENDIF
    NEXT j
NEXT i
 	
OUTPUT "Sorted array:"
FOR i ← 0 TO 4
    OUTPUT numbers[i]
NEXT i
	numbers = [42, 15, 8, 23, 4]
	n = len(numbers)
	
	# Outer loop controls number of passes
	for i in range(n - 1):
	    # Inner loop compares adjacent pairs
	    for j in range(n - 1 - i):
	        if numbers[j] > numbers[j + 1]:
	            # Standard 3-line swap using temp
	            temp = numbers[j]
	            numbers[j] = numbers[j + 1]
	            numbers[j + 1] = temp
	
	print("Sorted array:", numbers)

Question 8

Write a Pseudocode and Python program that sorts a list of 6 student test scores in **descending order** (highest to lowest) using an optimized Bubble Sort.
Pre-populated array: `scores = [65, 88, 72, 95, 60, 81]`
Your program must:

- Sort the list from highest to lowest.
- Include a boolean flag (e.g., `swapped`) to detect if any swaps occurred during a pass.
- Terminate the sorting process early using a conditional loop (`while`) if a complete pass is made with no swaps (meaning the list is already sorted).
- Output the final sorted scores and the total number of passes it took to sort.

Pseudocode	Python

DECLARE scores : ARRAY[0:5] OF INTEGER
DECLARE temp : INTEGER
DECLARE j : INTEGER
DECLARE passes : INTEGER
DECLARE swapped : BOOLEAN
DECLARE n : INTEGER
 	
scores[0] <- 65
scores[1] <- 88
scores[2] <- 72
scores[3] <- 95
scores[4] <- 60
scores[5] <- 81
 	
n <- 6
passes <- 0
swapped <- TRUE
 	
WHILE swapped = TRUE AND passes < n - 1
  swapped <- FALSE
  FOR j <- 0 TO n - 2 - passes
  // Check for descending order
  IF scores[j] < scores[j + 1] THEN
  temp <- scores[j]
  scores[j] <- scores[j + 1]
  scores[j + 1] <- temp
  swapped <- TRUE
  ENDIF
  NEXT j
  passes <- passes + 1
ENDWHILE
 	
OUTPUT "Sorted scores:"
FOR j <- 0 TO 5
  OUTPUT scores[j]
NEXT j
OUTPUT "Passes completed: ", passes
	scores = [65, 88, 72, 95, 60, 81]
	n = len(scores)
	passes = 0
	swapped = True
	
	# Continue passes until no swaps occur or all passes are completed
	while swapped and passes < n - 1:
	  swapped = False
	  for j in range(n - 1 - passes):
	  # Compare for descending order (highest to lowest)
	  if scores[j] < scores[j + 1]:
	  temp = scores[j]
	  scores[j] = scores[j + 1]
	  scores[j + 1] = temp
	  swapped = True
	  passes = passes + 1
	
	print("Sorted scores:", scores)
	print("Passes completed:", passes)

Question 9

Write a program that uses a subroutine (procedure) named `CalculateTotal` which takes an array of 5 integers as a parameter, uses a count-controlled loop to calculate the sum of all elements, and outputs the result.

Pseudocode	Python

`PROCEDURE CalculateTotal(Marks : ARRAY[0:4] OF INTEGER)`
`    DECLARE Total : INTEGER`
`    DECLARE Index : INTEGER`
 	
`    Total ← 0`
`    FOR Index ← 0 TO 4`
`        Total ← Total + Marks[Index]`
`    NEXT Index`
 	
`    OUTPUT "The total is: ", Total`
`ENDPROCEDURE`
 	
`// Main Program`
`DECLARE StudentMarks : ARRAY[0:4] OF INTEGER`
`DECLARE i : INTEGER`
 	
`StudentMarks[0] ← 12`
`StudentMarks[1] ← 18`
`StudentMarks[2] ← 15`
`StudentMarks[3] ← 20`
`StudentMarks[4] ← 14`
 	
`CALL CalculateTotal(StudentMarks)`
	`def calculate_total(marks):`
	`    total = 0`
	`    for mark in marks:`
	`        total = total + mark`
	`    print("The total is:", total)`
	
	`# Main Program`
	`student_marks = [12, 18, 15, 20, 14]`
	`calculate_total(student_marks)`

Question 10

Write a program that uses a subroutine (function) named `GetValidScore`. The function must:

- Accept two integer parameters: `MinScore` and `MaxScore`.
- Use a conditional loop (`WHILE` or `REPEAT...UNTIL`) to prompt the user to input an integer between `MinScore` and `MaxScore` inclusive.
- Output an error message and repeat the prompt if the input is outside this range.
- Return the valid score to the calling program.

The main program should call this function twice to get scores for Paper 1 (range 0 to 75) and Paper 2 (range 0 to 50), then calculate and output the total score.

Pseudocode	Python

`FUNCTION GetValidScore(MinScore : INTEGER, MaxScore : INTEGER) RETURNS INTEGER`
`    DECLARE Score : INTEGER`
 	
`    OUTPUT "Enter a score between ", MinScore, " and ", MaxScore, ":"`
`    INPUT Score`
 	
`    WHILE Score < MinScore OR Score > MaxScore`
`        OUTPUT "Invalid input. Please try again:"`
`        INPUT Score`
`    ENDWHILE`
 	
`    RETURN Score`
`ENDFUNCTION`
 	
`// Main Program`
`DECLARE Paper1 : INTEGER`
`DECLARE Paper2 : INTEGER`
`DECLARE TotalScore : INTEGER`
 	
`Paper1 ← GetValidScore(0, 75)`
`Paper2 ← GetValidScore(0, 50)`
 	
`TotalScore ← Paper1 + Paper2`
`OUTPUT "Combined Total Score: ", TotalScore`
	`def get_valid_score(min_score, max_score):`
	`    score = int(input(f"Enter a score between {min_score} and {max_score}: "))`
	
	`    while score < min_score or score > max_score:`
	`        print("Invalid input. Please try again.")`
	`        score = int(input(f"Enter a score between {min_score} and {max_score}: "))`
	
	`    return score`
	
	`# Main Program`
	`paper1 = get_valid_score(0, 75)`
	`paper2 = get_valid_score(0, 50)`
	
	`total_score = paper1 + paper2`
	`print("Combined Total Score:", total_score)`